

Processes

What is a process?

Structure of a process

Process state

Schedulers and Scheduling Queues

Context switching

Process creation and termination

Inter Process Communication

Dr. Manmath Narayan Sahoo

Department of Computer Science & Engineering

National Institute of Technology Rourkela

Odisha – 769008

Disclaimer – This slide is prepared taking content from various books and online resources. The presenter doesn't own the content. The inherited content is used for educational purpose only.

Process

- **Process** is a program that has initiated its execution.
- Program is *passive* entity stored on disk (**executable file**), process is *active*
 - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
 - Consider multiple users executing the same program

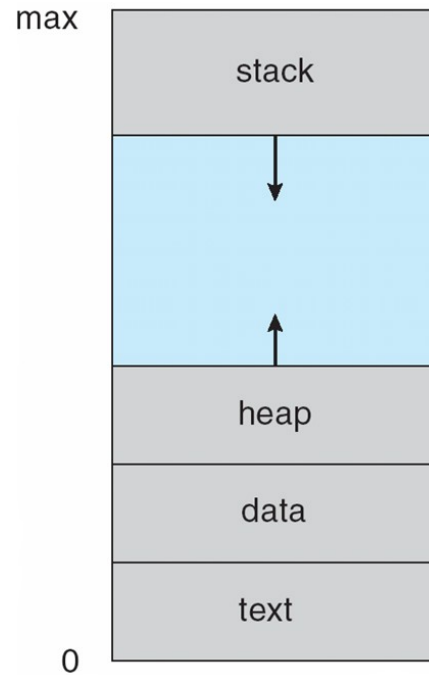
Process structure

A process includes:

- **text/code/program section** containing a copy of the machine code instructions
- Current **activity** including program counter, processor registers
[?] **system data section** storing process context and info about resources allocated
- **Stack** containing temporary data
 - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time

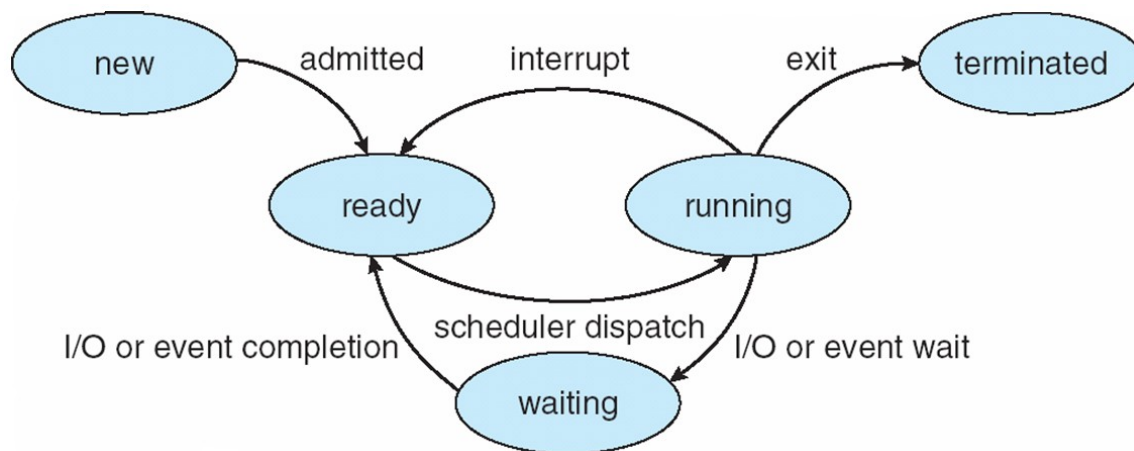
user and system data sections obviously may be different from one run of a process to another

Process in memory



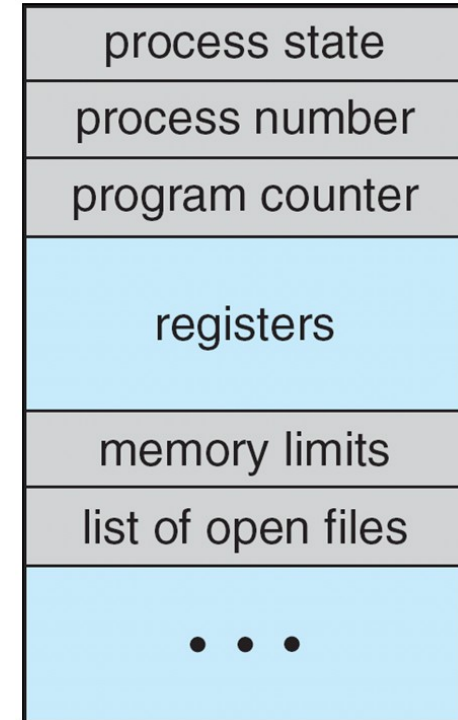
Process State

- As a process executes, it changes **state**
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution



Process Control Block (PCB)

- PCB - data structure that contains Information associated with each process:
 - Process ID
 - Process State
 - Program Counter
 - Register Contents
 - Starting address of the process
 - Size of the process
 - Pointer to Child Process
 - Pointer to Next PCB (Sibling Pointer)
 - Resource Pointer

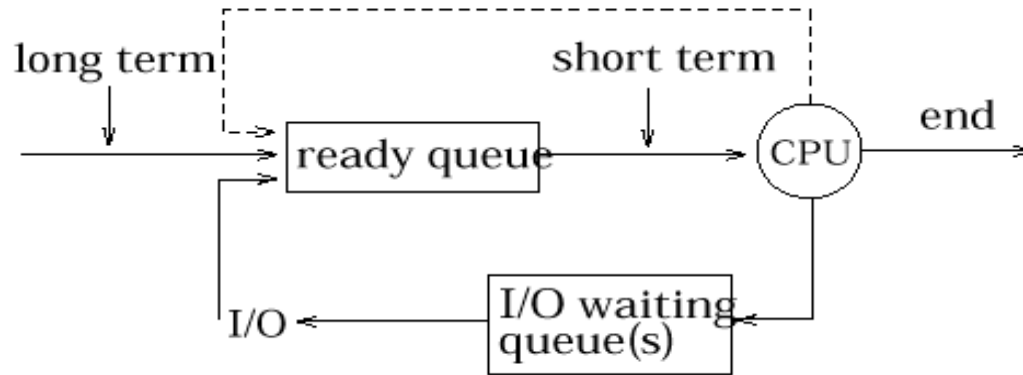


Process Scheduling Queues

- **Job queue** – Set of all processes just created (new state processes).
- **Ready queue** – set of all processes that are in main memory, ready to execute (ready state processes).
- **Device queues** – set of processes waiting to use an I/O device (waiting state processes).

Schedulers

- **Long-term scheduler (LTS)** (or job scheduler) [?] selects which processes should be brought into the ready queue.
- **Short-term scheduler (STS)** (or CPU scheduler) [?] selects which process should be executed next on CPU.

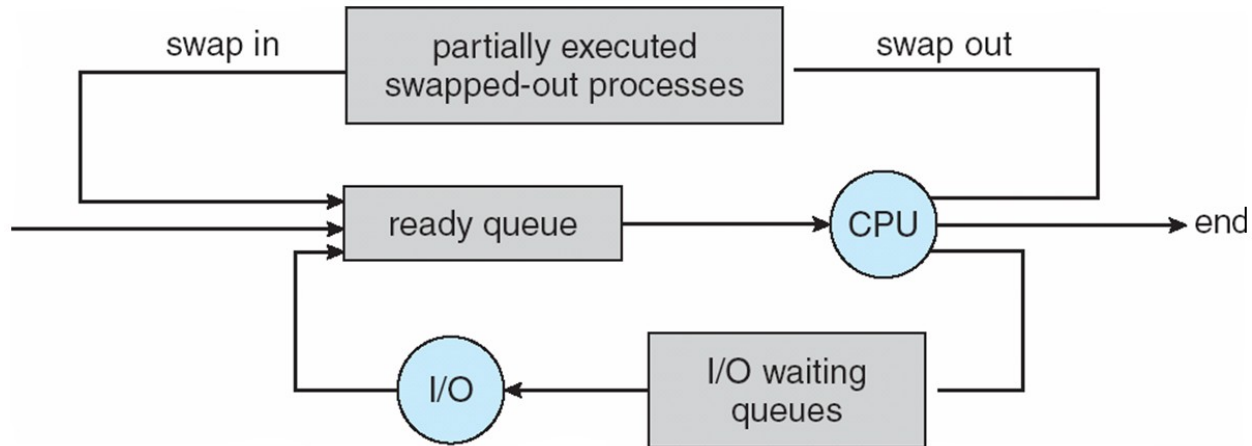


Schedulers

- Short-term scheduler is invoked very frequently (milliseconds) [?] (must be fast).
- Long-term scheduler is invoked very infrequently (seconds, minutes) [?] (may be slow).
- The long-term scheduler controls the *degree of multiprogramming*
- Long-term scheduler strives for good **process mix** of I/O-bound and CPU-bound processes
 - I/O-bound process [?] spends more time doing I/O than computations
 - CPU-bound process [?] spends more time doing computations
 - Only I/O bound processes [?] low CPU utilization
 - Only CPU-bound processes [?] low device utilization

Schedulers

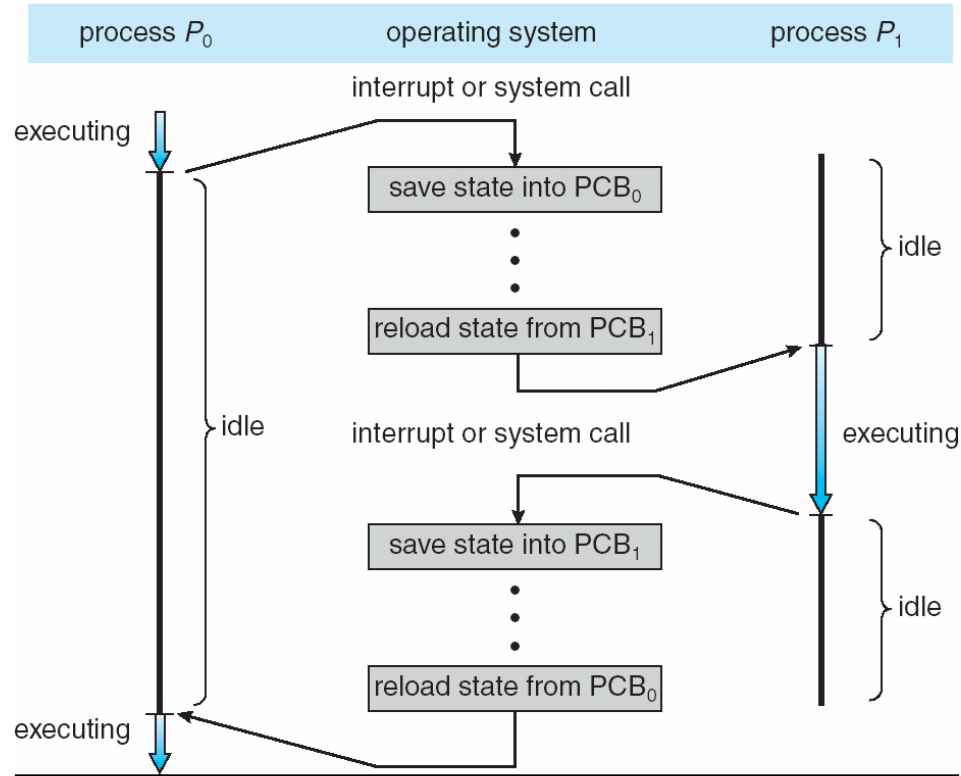
- Some OS have a light LTS; and a **Medium Term Scheduler (MTS)**.
- If degree of multiprogramming becomes high then total context switch overhead will be more.
- MTS can temporarily *swap* some processes out to disk and bring them back later.



Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching.

CPU switched between processes



Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- There must be a first process (**init** process) [?] this is a system process created during booting
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

A Tree of Processes in Linux

Process Termination

- Process executes last statement and asks the operating system to delete it using `exit()`.
 - Return data from child to parent.
 - Process' resources are deallocated by OS.
- Parent may terminate execution of children processes (**abort**). Reasons for this could be:
 - Task assigned to child is no longer required.
 - Parent is exiting and O/S does not allow children to continue if their parent terminates – **cascaded termination**.

Cooperating Processes

- Processes within a system may be *independent* or *cooperating*
- **Cooperating** process can legally affect or be affected by the execution of another process.

Reasons for cooperating processes:

- Processes may need to **share** data
 - More than one process reading/writing the same data (a shared file, a database record,...)
 - Output of one process being used by another
- **Ordering** executions of multiple processes may be needed to ensure correctness
 - Process X should not do something before process Y does something etc.
 - Need mechanisms to pass control signals between processes
- **Modularity**  dividing system functions into separate processes that then communicate
- Computation **speed-up**  but only if >1 processor

Inter-Process Communication (IPC)

Cooperating processes need **interprocess communication (IPC)**

- Message passing
 - P and Q exchange messages
- Shared memory
 - P writes into a shared location, Q reads from it and vice-versa

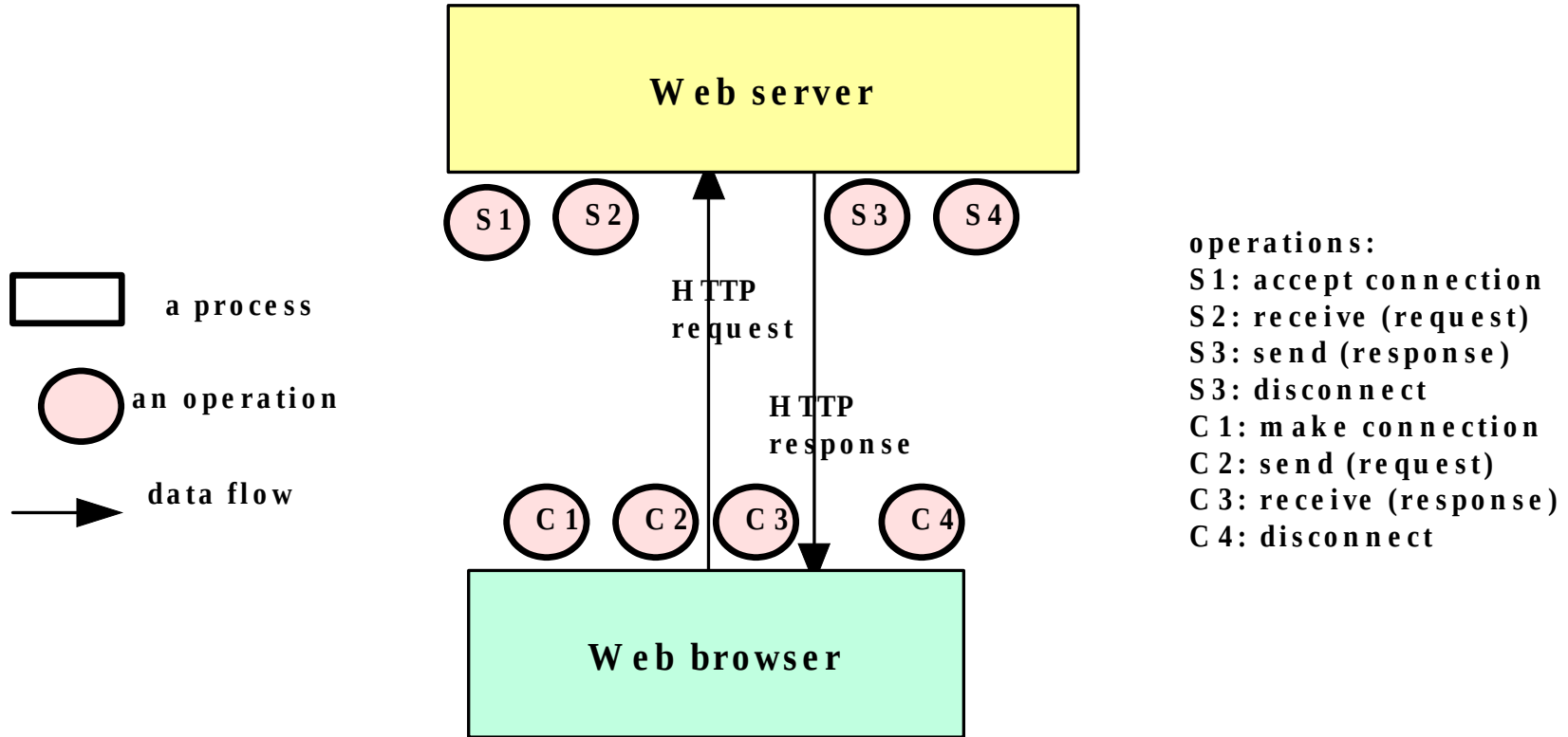
Inter-Process Communication (IPC)

(a) Message Passing

(b) Shared Memory



Message Passing System



Message Passing System

- Direct vs. Indirect
- Symmetric vs. Asymmetric
- Synchronous vs. Asynchronous

Message Passing System: Direct

Primitives:

- **Connect (sender_addr, receiver_addr)**
 - for connection-oriented communication.
- **Send ([receiver], message)**
 - e.g., Send(P, msg)
- **Receive ([sender], buffer)**
 - e.g., Receive(Q, buf)
- **Disconnect(connectionid)**
 - for connection-oriented communication.

Message Passing System: Direct

- **Symmetric** – Both the sender and receiver have to explicitly name each other.
- **Asymmetric** – Only the sender has to name the receiver, but the receiver doesn't need to name the sender.
 - Facilitates many-to-one communication.
 - `Send(P, msg);` [?] send message to P.
 - `Receive(id, mbuf);` [?] receive message from any process. “**id**” will be set to the name of the process with which communication has taken place.

Message Passing System: Indirect

- The messages are sent and received to/from **mailboxes** (also referred to as ports).
 - Each mailbox has a unique identifier.
 - Processes can communicate only if they share a mailbox
- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - send**(*A, message*) – send a message to mailbox A
 - receive**(*A, mbuffer*) – receive a message from mailbox A

Message Passing System: Indirect

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A
 - P_1 sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Message Passing System: Synchronous vs. Asynchronous

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - Blocking send [?]the sender is blocked until the message is received
 - Blocking receive [?]the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - Non-blocking send [?]the sender sends the message and continue
 - Non-blocking receive [?]the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible

Buffering during message passing

- Queue of messages attached to the link.
- Implemented in one of three ways
 - Zero capacity
 - No messages are queued on a link
 - Sender must wait for receiver, after generating one item
 - Bounded capacity
 - Finite length of n messages
 - Sender must wait if link full
 - Unbounded capacity
 - Infinite length
 - Sender never waits

Process: Exception Conditions

- Process Termination
 - Sender Terminates: If the receiver has initiated a blocking receive then the OS either terminates the receiver or notifies it that Sender has terminated
 - Receiver Terminates: If the sender has initiated a blocking send then the OS either terminates the Sender or notifies it that Receiver has terminated
- Lost Messages
 - The OS is responsible for detecting this event and for resending the message
 - The Sender is responsible for detecting this event and for resending it
 - The OS detects the event and notifies the Sender about it
- Scrambled Messages
 - Due to noise in the channel
 - Handled in the similar ways as lost messages